Task 3: Synchronisation

We have the Kuramoto model

$$\frac{d\theta_i}{dt} = \omega_i + \frac{K}{N}\sum_{j=1}^{N}\sin(\theta_j - \theta_i), \tag{1}$$

where N is the number of oscillators and ω_i is the frequency of oscillator i. ω is initiated as a random sample from the distribution

$$g(\omega) = \frac{\gamma}{\pi(\omega^2 + \gamma^2)}.$$

a)

We have the self-consistent equation

$$1 = K\int_{-\frac{\pi}{2}}^{\frac{\pi}{2}} \cos^2(\theta) g(Kr\sin\theta)d\theta. \tag{2}$$

By letting $r \to 0^+$ we get an expression for K_c (since the critical K is where r approaches zero from the right):

$$K_c = \frac{1}{g(0)\int_{-\frac{\pi}{2}}^{\frac{\pi}{2}}\cos^2(\theta)d\theta} = \frac{2}{\pi g(0)} = \left\{g(0) = \frac{1}{\pi\gamma}\right\} = 2\gamma$$

We now choose to look around the bifurcation ($r = 0$) by expanding $g(\omega)$ around zero:

$$g(\omega) = g(0) + g'(0)\omega + \frac{g''(0)}{2}\omega^2 + O(\omega^3).$$

With

$$\begin{aligned} g(0) &= \frac{1}{\pi\gamma}, \\ g'(\omega) &= -\frac{2\gamma\omega}{\pi(\omega^2+\gamma^2)^2} \implies g'(0) = 0, \text{ and} \\ g''(\omega) &= \frac{2\gamma(3\omega^2-\gamma^2)}{\pi(\omega^2+\gamma^2)^3} \implies g''(0) = -\frac{2}{\pi\gamma^3}, \end{aligned}$$

we get

$$g(\omega) = \frac{1}{\pi\gamma} - \frac{\omega^2}{\pi\gamma^3} + O(\omega^3).$$

We now choose to neglect the higher order terms and then plug our approximation into eq. 2:

$$\begin{aligned} 1 &= K\int_{-\frac{\pi}{2}}^{\frac{\pi}{2}} d\theta\cos^2(\theta)\left(\frac{1}{\pi\gamma} - \frac{K^2}{\pi\gamma^3}r^2\sin^2\theta\right) = K\left(\frac{1}{\pi\gamma}\int_{-\frac{\pi}{2}}^{\frac{\pi}{2}} d\theta\cos^2\theta - \frac{K^2r^2}{\pi\gamma^3}\int_{-\frac{\pi}{2}}^{\frac{\pi}{2}} d\theta\cos^2\theta\sin^2\theta\right) = \\ &= K\left(\frac{1}{\pi\gamma}\frac{\pi}{2} - \frac{K^2r^2}{\pi\gamma^3}\frac{\pi}{8}\right) = \frac{K}{2\gamma} - \frac{K^3r^2}{8\gamma^3}. \end{aligned}$$

Furthermore, we know that $r = C\sqrt{\mu}$, where $0 < \mu = (K - Kc)/Kc \ll 1$, around the bifurcation point. Thus, we can use the substitution $K = K_c(1+\mu) = 2\gamma(1+\mu)$:

$$1 = \frac{2\gamma(1+\mu)}{2\gamma} - \frac{8\gamma^3(1+\mu)^3r^2}{8\gamma^3} = (1+\mu) - (1+\mu)^3r^2 \implies r^2(1+\mu)^3 = \mu. \tag{3}$$

Since we are looking for a solution where $r^2 \sim \mu$, we can neglect all terms with $O(\mu)$ in $(1+\mu)^3$ and approximate eq. 3 as

$$r^2 = \mu \implies r = \sqrt{\mu} \implies C = 1$$

for small variations around the bifurcation point where $r \to 0^+$.

b)

From the analysis above we obtained the critical coupling constant $K_c = 2\gamma$. In the simulation we let $\gamma = 1 \implies K_c = 2$, turning the frequency distribution $g(\omega)$ into the standard Cauchy (or Lorentz) distribution. To simulate, we first initialize the phases, θ_i, and frequencies, ω_i, for the N oscillators. Then, each phase is updated by integrating 1 numerically using Euler explicit integration for each oscillator i. The order parameter is calculated using

$$\vec{r} = \frac{1}{N}\sum_{j=0}^{N} e^{i\theta_j}$$

and then taking the absolute value $r = |\vec{r}|$. The simulation is run for $K = \{1, 2.1, 4\}$ and $N = \{20, 100, 300\}$ with a timestep of $dt = 0.001$s and for a total time of $T = 50$s.

In the figure below we see r as a function of time during the simulations. As we can see, the number of oscillators seems to have an effect on the amplitude of the oscillations of r, with lower oscillations for large N. If we look at $N = 300$, we can see that for $K = 1$ (well below the critical value of $K_c = 2$), r is close to 1. This suggests that coupling is too weak for any notable synchronisation to happen, which is in line with our theoretical analysis.

Furthermore, at $K = 2.1$ (slightly above K_c), we see r rising slightly, suggesting that the bifurcation point has been reached. In our approximation around the bifurcation point, we concluded that $r = \sqrt{\mu} = \sqrt{(K - K_c)/K_c}$. With $K = 2.1$, we obtain a theoretical approximation of $r = \sqrt{0.05} \approx 0.22$. While not exactly the same, this approximation is relatively close to our simulated mean value (for $K = 2.1$, $N = 300$) of $r_m = 0.15$. At $K = 4 > K_c$, we, as expected, see a higher value of r, which means that a higher degree of synchronisation is happening.

In conclusion, we have seen that the mean field theory seems to work in practice. Had we used an even higher number of oscillators, we probably would have come closer to our theoretical hypothesis, but at the cost of longer computing time.

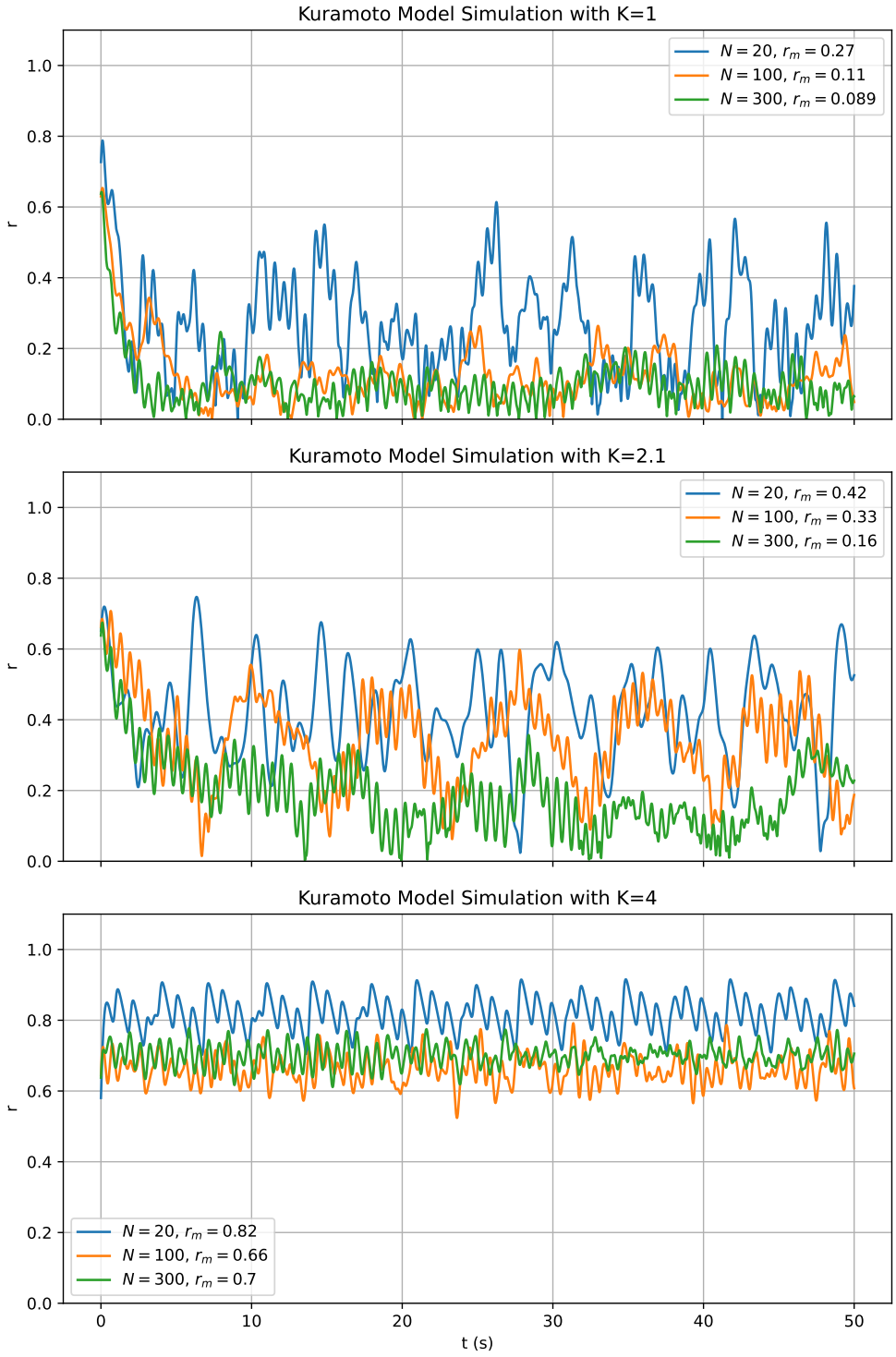

Figure 1: **Simulation of Kuramoto model for different values K.** Plots show the order parameter r as a function of time, and for different number of oscillators. r_m in the legends are means calculated for 10 s $< t <$ 50 s.

Python code

```
import numpy as np
import matplotlib.pyplot as plt

class KuramotoModelSimulation:
    def __init__(self, N: int, K: float, dt: float) -> None:
        self.N: int = N
        self.K: float = K
        self.dt: float = dt

        self.thetas: np.ndarray = np.random.uniform(-np.pi/2, np.pi/2, size=self.N)
        self.omegas: np.ndarray = np.random.standard_cauchy(size=self.N) # Mean=0 and gamm
        self.omegas = np.clip(self.omegas, -10, 10) # Prevent really large omegas

    def get_r_vec(self) -> np.ndarray:
        """
        r = 1/N * sum(e^(i*theta_j))
        """
        xs = np.cos(self.thetas)
        ys = np.sin(self.thetas)

        r_x = np.sum(xs) / self.N
        r_y = np.sum(ys) / self.N

        r_vec = np.array([r_x, r_y])

        return r_vec

    def calc_r(self) -> np.floating:
        r_vec = self.get_r_vec()
        return np.linalg.norm(r_vec)

    def dtheta_dt(self) -> np.ndarray:
        theta_diffs = self.thetas[None, :] - self.thetas[:, None]

        return self.omegas + self.K / self.N * np.sum(np.sin(theta_diffs), axis=1)

    def update(self) -> None:
        """
        Euler explicit numerical integration:

        y_(i+1) = y_i + dt*f'(t, y_i)
        """
        self.thetas += self.dt * self.dtheta_dt()
        self.thetas = np.mod(self.thetas, 2*np.pi)

# Run simulation
dt = 0.001
T_tot = 50
Ks = [1, 2.1, 4]
Ns = [20, 100, 300]

data_K_N = []

for K in Ks:
```

```
    data_K = []

    for N in Ns:
        model = KuramotoModelSimulation(N, K, dt)

        ts = np.linspace(0, T_tot, int(T_tot/dt))
        rs = [model.calc_r()]

        for i in range(len(ts)-1):
            model.update()
            rs.append(model.calc_r())

        data_K.append(rs)

    data_K_N.append(data_K)

# Find mean r and variance
r_means = []
r_vars = []

for data_K, K in zip(data_K_N, Ks):
    print(f"K={K}:")

    r_means_K = []
    r_vars_K = []

    for data_N, N in zip(data_K, Ks):
        # Include data from after 10s simulation time
        r_mean = np.mean(data_N[int(10/dt):])
        r_var = np.var(data_N[int(10/dt):])

        print(f"    N={N}: r_mean={r_mean:.2}, r_var={r_var:.2}")

        r_means_K.append(r_mean)
        r_vars_K.append(r_var)

    r_means.append(r_means_K)
    r_vars.append(r_vars_K)

# Plotting
fig, axes = plt.subplots(3, 1, sharex=True, figsize=[8, 12])

for data_K, r_means_K, K, ax in zip(data_K_N, r_means, Ks, axes):

    for data_N, r_mean, N in zip(data_K, r_means_K, Ks):
        ax.plot(ts, data_N, label=f"$N={N}$, $r_m={r_mean:.2}$")

    ax.set_title(f"Kuramoto Model Simulation with K={K}")
    ax.set_ylabel("r")
    ax.set_ylim(0, 1.1)
    ax.legend()
    ax.grid()
```

```
ax.set_xlabel("t (s)")
plt.tight_layout()
plt.savefig("2_3.pdf")
plt.show()
```